МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

# Лабораторная работа № 4

Дисциплина: «Аппаратные средства телекоммуникационных систем» Тема: «Функциональная декомпозиция с использованием глобальной суммы и критических секций»

Работу выполнил
Студент гр. 4404
Семенов М.А.

Казань, 2026

**1. Цель работы**

Изучить механизм критических секций в OpenMP для безопасного доступа к разделяемым (глобальным) переменным. Закрепить навык функциональной декомпозиции задачи на примере вычисления среднего значения по формуле (Σx + Σy + Σz) / 3N.

**2. Задание**

**Задача 6.** Используя функциональную декомпозицию, перепишите задачу 5 для вычисления значения по формуле: (Σx + Σy + Σz) / 3N, где N – количество точек. Используйте глобальную сумму и критические секции.

**Общие требования:**

- Число потоков передается через командную строку.
- Использовать функциональную декомпозицию (sections).
- Обязательное применение глобальной суммы и критических секций (critical).

**3. Ход работы**

**3.1. Логика работы**

Программа читает координаты 1 000 000 точек из текстового файла. В отличие от предыдущей работы, где вычислялись три отдельных средних значения, здесь требуется найти единое число: сумму всех координат, деленную на утроенное количество точек.

**3.2. Функциональная декомпозиция**

Задача разбивается на три параллельные секции (#pragma omp parallel sections). Каждая секция отвечает за вычисление суммы координат по своей оси:

- Первая секция считает Σx.
- Вторая секция считает Σy.
- Третья секция считает Σz.

Для минимизации времени нахождения в критической секции, каждый поток сначала накапливает сумму в локальной переменной (local_sum), и только потом прибавляет ее к глобальной.

**3.3. Работа с глобальной суммой и критической секцией**

Для накопления общего результата используется разделяемая переменная global_sum. Чтобы избежать состояния гонки (когда несколько потоков одновременно пишут в одну ячейку памяти), операция прибавления защищена критической секцией:

```
#pragma omp critical
{
   global_sum += local_sum;
}
```

Директива critical гарантирует, что в каждый момент времени операцию сложения выполняет только один поток. Остальные потоки в это время ожидают своей очереди.

**3.4. Проверка корректности**

После завершения работы всех потоков вычисляется итоговый результат по формуле global_sum / (3.0 * N). Для проверки этот результат сравнивается с

последовательным вычислением. Если разница не превышает 1e-6, программа выводит Check: OK.

**4. Результаты работы**

```
C:\Users\Admin\source\repos\lab4_1\lab4_1>.\lab4.exe 1 points.txt
File not found. Generating points.txt with 1000000 points...
Loaded 1000000 points from points.txt
Threads: 1

--- Functional Decomposition + Critical Section ---
Time: 0.0079385 sec
Global sum: 1.345e+07
Result (?x+?y+?z)/3N = 4.48333

Check sequential value: 4.48333
Check: OK

C:\Users\Admin\source\repos\lab4_1\lab4_1>.\lab4.exe 2 points.txt
Loaded 1000000 points from points.txt
Threads: 2

--- Functional Decomposition + Critical Section ---
Time: 0.0054012 sec
Global sum: 1.345e+07
Result (?x+?y+?z)/3N = 4.48333

Check sequential value: 4.48333
Check: OK

C:\Users\Admin\source\repos\lab4_1\lab4_1>.\lab4.exe 4 points.txt
Loaded 1000000 points from points.txt
Threads: 4

--- Functional Decomposition + Critical Section ---
Time: 0.0029894 sec
Global sum: 1.345e+07
Result (?x+?y+?z)/3N = 4.48333

Check sequential value: 4.48333
Check: OK

C:\Users\Admin\source\repos\lab4_1\lab4_1>.\lab4.exe 8 points.txt
Loaded 1000000 points from points.txt
Threads: 8

--- Functional Decomposition + Critical Section ---
Time: 0.0046723 sec
Global sum: 1.345e+07
Result (?x+?y+?z)/3N = 4.48333

Check sequential value: 4.48333
Check: OK
```

**Сводная таблица измерений:**

| Число потоков | Время выполнения, сек | Результат (Σx+Σy+Σz)/ 3N | Проверка (Check) |
|---|---|---|---|
| **1** | 0.007938 | 4.48333 | OK |
| **2** | 0.005401 | 4.48333 | OK |
| **4** | 0.002989 | 4.48333 | OK |
| **8** | 0.004672 | 4.48333 | OK |

**5. Анализ результатов**

В ходе всех запусков результат вычислений оставался неизменным (4.48333), а проверка выдавала Check: OK. Это доказывает, что использование критических секций обеспечило корректное суммирование глобальной переменной без гонок данных.

**Анализ производительности:**

1. На 1 и 2 потоках время закономерно снижается (с 0.0079 до 0.0054 сек), так как нагрузка распределяется между потоками.
2. На 4 потоках достигнуто наилучшее время (0.0029 сек), что близко к идеальному масштабированию.
3. **На 8 потоках время выполнения выросло (до 0.0046 сек).** Это объясняется ограничением функциональной декомпозиции: в программе всего 3 независимые секции (по осям X, Y, Z). При запуске на 8 потоках три потока выполняют полезную работу, а остальные пять простаивают. При этом накладные расходы OpenMP на создание 8 потоков, распределение задач и синхронизацию критических секций становятся больше, чем выигрыш от параллелизма, что и приводит к замедлению.

---

**6. Выводы**

В ходе работы были изучены критические секции (#pragma omp critical) как средство синхронизации потоков при работе с разделяемыми ресурсами.

Экспериментально подтверждено, что функциональная декомпозиция позволяет распараллелить разнородные участки кода, однако её максимальная эффективность ограничена количеством логических блоков (в данном случае — тремя). Также было выяснено, что при использовании глобальных переменных необходимо защищать операции записи критическими секциями, чтобы избежать искажения результатов, но при этом важно следить за тем, чтобы время нахождения в критической секции было минимальным (для этого использовались локальные аккумуляторы).